

Step 3 — A Retriever Agent That Judges Its Own Search Results — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume `retriever.py`'s `retrieve(query: str, k: int = 3) -> list[Document]` is reused as-is from `08_rag`, backed by a small Chroma collection of a few short documents about one topic you know well.

Basic Version

Approach 1 — "did anything come back" as the relevance check

```
# agents/retriever_agent.py
from retriever import retrieve

def retrieve_node(state):
    chunks = retrieve(state["task"], k=3)
    grounded = len(chunks) > 0
    return {"found_chunks": chunks, "grounded": grounded}
```

```
# graph.py (excerpt)
def route_after_retrieve(state):
    return "answer" if state["grounded"] else "cant_ground"

builder.add_node("retrieve", retrieve_node)
builder.add_conditional_edges("retrieve", route_after_retrieve, {"answer": "answer", "cant_ground": "cant_ground"}
```

Expected output, run against an in-scope question:

```
grounded=True, 3 chunks found
```

Expected output, run against a question about a topic wildly outside the knowledge base, worded generically ("Tell me about something else"):

```
grounded=True, 3 chunks found    <-- wrong! the vector store still returns its 3 *
```

This is the exact trap the README warns about: `retrieve()` almost always returns *something*, because a vector search returns the closest matches it has, not "nothing" when nothing is truly close. `len(chunks) > 0` will be `True` for nearly every query, which means this "judgment" never actually catches an ungrounded case — it only catches the rare situation where the knowledge base is completely empty.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a real judgment call, plus conditional routing wired end to end

```
# state.py
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages
from langchain_core.documents import Document
```

```
class GraphState(TypedDict):
    task: str
    messages: Annotated[list, add_messages]
    found_chunks: list[Document]
    grounded: bool
```

```
# agents/retriever_agent.py
from langchain_openai import ChatOpenAI
from retriever import retrieve

def judge_relevance(task: str, chunks: list) -> bool:
    if not chunks:
        return False
    combined = "\n---\n".join(c.page_content for c in chunks)
    prompt = (
        f"Question: {task}\n\nRetrieved text:\n{combined}\n\n"
        "Does this text actually contain material that answers the question? Reply with 'yes' or 'no'."
    )
    model = ChatOpenAI(model="gpt-4o-mini")
    answer = model.invoke(prompt).content.strip().lower()
    return answer.startswith("yes")

def retrieve_node(state):
    chunks = retrieve(state["task"], k=3)
    grounded = judge_relevance(state["task"], chunks)
    return {"found_chunks": chunks, "grounded": grounded}
```

```
# graph.py
def needs_search(state):
```

```
# simple heuristic for this exercise: assume most tasks in this project need
# a real system might check for keywords, or always route through search and
# judge_relevance handle "nothing relevant" instead of guessing up front
return "search"

def route_after_retrieve(state):
    return "answer" if state["grounded"] else "cant_ground"

builder.add_conditional_edges(START, needs_search, {"search": "retrieve", "skip":
builder.add_node("retrieve", retrieve_node)
builder.add_conditional_edges("retrieve", route_after_retrieve, {"answer": "answe
```

Expected output, in-scope question:

```
grounded=True
```

Expected output, an honestly out-of-scope question ("What's the boiling point of mercury?" against a knowledge base about a company's return policy):

```
grounded=False
```

Difference from Basic: the relevance check now actually reads the retrieved text against the question, instead of just checking the list isn't empty — this is what catches "found 3 chunks, none of them actually relevant," not just "found 0 chunks." `needs_search` is still a placeholder returning `"search"` unconditionally — real conditional skipping is what Advanced adds.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — the rubric written down, and a real "does it need search" check

```
# agents/retriever_agent.py
"""
Relevance rubric (decided before writing judge_relevance, not adjusted after
seeing test output): a search result is "grounded" only if at least one
returned chunk specifically mentions the exact entity or fact the question
asks about – not merely the same general topic area.
"""

def judge_relevance(task: str, chunks: list) -> bool:
    if not chunks:
        return False
    combined = "\n--\n".join(c.page_content for c in chunks)
    prompt = (
        f"Question: {task}\n\nRetrieved text:\n{combined}\n\n"
        "Does this text specifically mention the exact thing being asked about "
        "(not just the same general topic)? Reply only yes or no."
    )
    model = ChatOpenAI(model="gpt-4o-mini")
    answer = model.invoke(prompt).content.strip().lower()
    return answer.startswith("yes")

def retrieve_node(state):
    chunks = retrieve(state["task"], k=3)
    grounded = judge_relevance(state["task"], chunks)
    return {"found_chunks": chunks, "grounded": grounded}
```

```
# graph.py – a real needs_search check, not an unconditional "search"
GREETING_WORDS = {"hi", "hello", "thanks", "thank you", "ok", "okay"}

def needs_search(state):
    task_lower = state["task"].strip().lower()
    if task_lower in GREETING_WORDS:
        return "skip"
    return "search"
```

Approach 2 — the 2 tests the README's Step 3 explicitly asks for

```
# test_project3.py
from graph import build_graph

def test_in_scope_question_grounds():
    graph = build_graph()
    result = graph.invoke({
        "task": "What is the return window for a damaged item?", # genuinely in
        "messages": [],
    })
    assert result["grounded"] is True
    assert len(result["found_chunks"]) > 0

def test_out_of_scope_question_does_not_ground():
    graph = build_graph()
    result = graph.invoke({
        "task": "What is the boiling point of mercury in Celsius?", # genuinely
        "messages": [],
    })
    assert result["grounded"] is False
```

Expected output, running both tests:

```
test_in_scope_question_grounds PASSED
test_out_of_scope_question_does_not_ground PASSED
```

Both tests passing together is what actually proves `judge_relevance` discriminates — either test passing alone (especially just the first) proves much less; a relevance check that always returns `True` would still pass `test_in_scope_question_grounds`.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's `judge_relevance` is real, but `needs_search` is a stub that never actually skips anything, and there's no test proving the "can't ground this" path is reachable at all. Approach 1 makes `needs_search` a genuine decision and writes the relevance rubric down as a comment before the code that implements it. Approach 2 is the concrete proof — 2 tests, one for each routing outcome — that the README's Step 3 checklist asks for by name ("confirm the 'can't ground this' path actually runs").

Which one should you actually write? Both. The rubric-as-comment habit costs one paragraph and makes it obvious, 3 months later, what "grounded" was actually supposed to mean when someone (possibly you) is debugging why a question got routed the "wrong" way. The 2-test pair in

Approach 2 is non-negotiable — a relevance judge with no test proving it can say "no" is functionally identical to Basic's `len(chunks) > 0`, just with extra API calls.